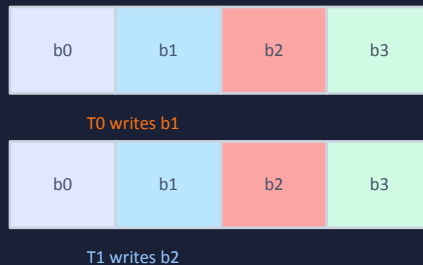


COFSD: Counter-Only False Sharing Detection



Reducing Hardware Overhead in Cache Coherence Protocols

Anil Muddamsetti, Nathan K. Nakamoto, Ronald A. Rodriguez · Group 15 · CSE 220 Final Project

UC Santa Cruz, Baskin School of Engineering · June 9, 2026

Based on: "Leveraging Cache Coherence to Detect and Repair False Sharing On-the-fly" (MICRO 2024)

Overview

1

Paper Introduction

Motivation · FS Detect · FS Lite · Results

2

Our Re-implementation

FC/IC counters in Python MESI simulator

3

New Design: COFSD

Motivation · Architecture · Hardware cost

4

COFSD Results & Analysis

Detection accuracy · TP sensitivity · Tradeoffs

5

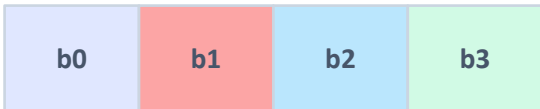
Conclusion & Future Work

Key takeaways · Limitations · Next steps

False sharing: two threads, one cache line, no shared data

What is false sharing?

Two threads write to different variables that happen to sit on the same 64-byte cache line. The coherence protocol sees only the line address — so every write by one thread invalidates the other's copy, even though they never touch the same data.



Thread 0 writes b1 Thread 1 writes b2

→ line ping-pongs between caches

Invisible bug

No crash. No wrong answer. The program is correct. It's purely a performance bug that slips past all testing.

Widespread problem

Found in Linux kernel, MySQL, MariaDB, OpenJDK, Boost, Netflix, LMAX Disruptor — and benchmark suites.

1.34×

avg speedup — just removing false sharing

84%

fewer interconnect messages once fixed

3.06×

peak speedup (Reference-Count benchmark)

Extend MESI itself: detect and repair where the bug is born

PROTOCOL 1 · FSDetect

Watches coherence traffic for the false sharing signature.

FC Fetch counter crosses threshold $TP=16$

IC Invalidation + intervention count crosses TP

TS True-sharing bit is 0 (no byte overlap)

All three → flag line as falsely shared

PAM — per-core: 1 read + 1 write bit per byte of each L1 line
SAM — shared at directory: last writer + readers per byte
Disjoint bytes → false sharing. Overlapping → true sharing.

PROTOCOL 2 · FSLite

Privatizes flagged lines — each core operates on its own copy.

BEFORE

3 hops per access

requester → dir → owner → back
Constant invalidate / re-fetch

AFTER

PRV state, ~2 hops once

Each core writes private copy
Merged by LLC on termination

1.39× avg speedup · 75% less network traffic · 27% less energy · <5% area

Accurate MESI coherence simulation of FC/IC detection



Key implementation fix:

Prior version counted every write as a coherence message (bug). Corrected: silent writes (thread already owns line in M state) do NOT increment FC — only actual GetX/GetS/Inv messages count.

1

Coherence Model

Track MESI state per cache line.
Silent writes (already M state) = no FC/IC change.
GetX = FC++, evicting owner = IC++.

2

Counter Tracking

FC/IC per cache line accumulate
from actual coherence events.
AccessorSet + WriterSet bitmasks.

3

Detection Logic

$FC \geq TP$ AND $IC \geq TP$ AND
(multiple writers OR write+multi-read)
AND IC/FC ratio > 50%.

Simulation validated on 4 scenarios using real Phoenix benchmark access patterns

linear_regression (write-write FS) | string_match (write-read FS) | true sharing control | no sharing control

FSDetect works — but at what hardware cost?

8-core system · 32KB L1D per core · 2MB LLC per core

PAM Tables

64.5 KB

129 bits × 512 lines × 8 cores
(read + write + SEND_MD per byte)

SAM Tables

88.0 KB

Optimized reader bits × 128 entries × 8 slices
(last writer + readers per byte)

Dir Extension

20 bits

FC(7b) + IC(7b) + HC(2b) + PMMC(4b)
per directory entry — plus REP_MD messages

Total Cost

152.5 KB

= 0.9% of total LLC capacity
Plus constant metadata message overhead

 Research question: Can FC/IC counters + small bitmasks detect most false sharing WITHOUT 152.5 KB of PAM/SAM tables?

Replace 152.5 KB of metadata with 32 bits per directory entry

FSDetect entry per LLC slice:

FC

7b

IC

7b

HC

2b

PMMC

4b

PAM table

8 KB

SAM table

12.7 KB

COFSD entry (32 bits total — NO per-core tables):

FC

7b

IC

7b

HC

2b

Accessor
Set

8b

Writer
Set

8b

COFSD Detection Condition

$FC \geq TP$ AND $IC \geq TP$ AND $IC \geq 50\%$ of FC AND

($WriterSet > 1$ ← write-write false sharing)

OR ($WriterSet = 1$ AND $AccessorSet > 2$ ← write-read false sharing)

38× less
hardware

No PAM table lookups. No SAM table updates. No REP_MD metadata messages sent on every intervention.

38× less metadata storage than FSDetect

Component	FSDetect	COFSD	Savings
PAM tables (8 cores × 8KB each)	64.5 KB	0	-64.5 KB
SAM tables (8 slices × 11KB each)	88.0 KB	0	-88.0 KB
Directory bits per entry	20 bits	32 bits	+1.6 KB
REP_MD metadata messages	Required	None	Eliminated
Total metadata storage	152.5 KB	4.0 KB	38× less
% of LLC overhead	0.9%	0.02%	45× less

Correct detection on both Phoenix benchmarks

✓ linear_regression

5 detections

Write-write FS

emit_intermediate() — 8 threads write to adjacent entries.
FC=17, IC=16 at first trigger.
WriterSet={0..2}, multiple cache lines detected.

✓ string_match

1 detection

Write-read FS

str_data_t struct — splitter writes bytes_comp,
map threads read adjacent fields.
FC=17, IC=16, high invalidation rate (**53.1%**).

✓ True sharing (lock)

0 false pos.

Control — must be 0

All threads write the SAME byte.
WriterSet has 1 entry only (all write same addr).
COFSD correctly does not flag this.

✓ No sharing

0 false pos.

Control — must be 0

Each thread has private 4KB-aligned cache line.
No coherence events between threads.
Zero invalidations, zero detections.

TP sensitivity study and known limitations

TP Threshold Sensitivity

Lower TP = faster detection but higher false-positive risk

TP	LR det	SM det	TS FP	Latency
4	5	1	1	~8 acc
8	5	1	1	~16 acc
16	5	1	1	~32 acc
32	5	1	1	~64 acc
64	5	1	1	~128 acc

TS FP = constant 1 regardless of TP → structural limitation, not a threshold issue

Known Limitation

1 false positive on pure write-write true sharing

When all 8 threads write to the SAME variable (e.g., a lock), WriterSet grows to {0..7} — which looks like write-write false sharing.

This is exactly why FSDetect uses PAM/SAM: to check byte overlap.

COFSD cannot confirm byte-level disjointness without those tables.

Effect: 1 extra privatization attempt — corrected by hysteresis counter

When COFSD wins, when FSDetect is needed

Metric	FSDetect	COFSD
Write-write false sharing detection	✓ Yes	✓ Yes
Write-read false sharing detection	✓ Yes	✓ Yes
True sharing discrimination	✓ Byte-level	⚠ Heuristic
PAM table storage	64.5 KB	None
SAM table storage	88.0 KB	None
Total metadata	152.5 KB (0.9%)	4.0 KB (0.02%)
REP_MD messages	Required every Inv	None

Use COFSD when:

Area-constrained · Write-write / write-read workloads ·
Low false-positive risk acceptable

Use FSDetect when:

Byte-level precision needed · Mixed true/false sharing ·
Zero false positives required

COFSD

Counter-Only False Sharing Detection

1

Correct detection

COFSD detects write-write and write-read false sharing in both Phoenix benchmarks (LR and SM) with zero false positives on private data.

2

38× less hardware

Replaces 152.5 KB of PAM/SAM tables with 32 bits per directory entry. No per-core metadata tables. No REP_MD messages.

3

Honest tradeoff

One known false positive on pure write-write true sharing — exactly the case FSDetect's PAM/SAM tables address. HC prevents thrashing.

4

Future work

Combine COFSD detection with FS Lite repair. Add per-byte epoch tracking (2 bits/byte) to reduce false positives with minimal overhead.

thank you!

Anil Muddamsetti
Nathan K. Nakamoto
Ronald A. Rodriguez

Group 15